

Programação Orientada a Objetos

Capítulo 5 — Linguagem de Programação Java

Aula 05

O que veremos hoje?

- 1 *Java: história, portabilidade e JVM*
- 2 Criação de classes e o método `main`
- 3 Métodos: `void`, retorno e comentários
- 4 Modificadores de acesso (encapsulamento)
- 5 Construtores múltiplos (polimorfismo)
- 6 **Superclasses e Subclasses (herança) + Dinâmicas**



5.1 Linguagem *Java*

Da Sun Microsystems ao mundo: portabilidade com a JVM

Origem e estratégia do *Java*

- Investimento inicial da *Sun Microsystems* — visão de mercado para dispositivos inteligentes;
- Estratégia de popularização: sintaxe baseada na linguagem *C* (a mais popular da época);
- Programadores migraram facilmente — comandos familiares, curva de aprendizado reduzida;
- 1995: *Sun* apresenta *Java* para grandes empresas, *World Wide Web* e dispositivos de consumo;
- Hoje é uma das linguagens mais populares do mundo, com vasta comunidade e ecossistema.

Compilação + Interpretação: o papel da JVM

- Java usa modelo híbrido: código é compilado em *bytecode* (intermediário) e interpretado pela JVM.
- JVM (*Java Virtual Machine*): máquina virtual instalada no equipamento que interpreta o bytecode.
- Compilado (ex.: C): melhor desempenho, mas sem portabilidade entre SOs.
- Interpretado (ex.: Python): portátil, mas mais lento (interpretado a cada execução).
- Java: meio-termo — *bytecode* compilado uma vez, JVM interpreta em qualquer plataforma.
- Fluxo: Código Java → Compilador → Bytecode (.class) → JVM
Windows/Linux/Mac → Execução.

Dinâmica 1 — Compilado vs. Interpretado vs. Java

- Em duplas, preenchem a tabela comparativa abaixo:
 - Colunas: Compilado (*C*) | Interpretado (*Python*) | Híbrido (*Java*);
 - Linhas: Desempenho | Portabilidade | Necessita *JVM*? | Exemplo de uso ideal;
 - Discutam: por que Java escolheu o modelo híbrido? Quais as vantagens práticas?
- Tempo: 5 minutos.

Respostas — Dinâmica 1

- *C* (compilado): alto desempenho, baixa portabilidade, não usa *JVM*. Ideal: sistemas operacionais, *drivers*;
- *Python* (interpretado): desempenho menor, alta portabilidade, não usa *JVM*. Ideal: *scripts*, *IA*, prototipagem;
- *Java* (híbrido): bom desempenho, alta portabilidade, requer *JVM*. Ideal: *apps* corporativos, *Android*, *web*;
- *Java* escolheu o modelo híbrido para ter o lema '*Write Once, Run Anywhere*' (escreva uma vez, rode em qualquer lugar).



5.1 Classes em *Java*

Criando a estrutura base no NetBeans

Criando uma classe Java no *NetBeans*

- Ao criar um novo projeto, a *IDE* gera automaticamente a classe principal com o método `main`;
- O método `main` é o ponto de entrada — primeira coisa executada ao iniciar o programa;
- Para criar nova classe: botão direito no pacote → Nova → Classe Java → nomear a classe;
- Convenção: nomes de classe sempre iniciam com letra maiúscula (ex.: Carro, Pessoa, ContaBancaria);
- Escopo delimitado por `{ }` — tudo entre chaves pertence àquele bloco.

Exemplo: classe Carro em Java

```
public class Carro {  
    // Atributos  
    public float motor;  
    public String modelo;  
    public int ano;  
  
    // Método sem retorno (void)  
    public void ligar() {  
        System.out.println("Carro Ligou!");  
    }  
  
    // Método com retorno (int)  
    public int calcularAno(int ano) {  
        return this.ano;  
    }  
}
```

Atributos definem o estado; métodos definem as ações do objeto.

Dinâmica 2 — Crie sua Primeira Classe

- Individualmente, escreva no papel (ou no caderno) uma classe *Java* para 'Aluno';
- Inclua: pelo menos 3 atributos (`nome`, `matricula`, `nota`), 1 método `void` e 1 método com retorno;
- Lembre-se: classe com letra maiúscula, `{ }` para delimitar escopo, tipos primitivos nos atributos;
- Método `void`: `exibirDados()` → imprime `nome` e `matrícula`;
- Método com retorno: `verificarAprovacao()` → retorna `true` se `nota` ≥ 6 ;
- Tempo: 6 minutos.

Exemplo de resposta — Dinâmica 2

```
public class Aluno {  
    public String nome;  
    public int matricula;  
    public float nota;  
    public void exibirDados() {  
        System.out.println(nome + " - " + matricula);  
    }  
    public boolean verificarAprovacao() {  
        return this.nota >= 6;  
    }  
}
```

- `void` = não retorna nada, apenas executa.
- `boolean` = retorna `true/false` com `'return'`.



5.2 Métodos em *Java*

Ações dos objetos: void, retorno e comentários

Tipos de métodos e comentários

- Métodos são ações que os objetos realizam — definidos dentro da classe;
- Método `void`: não retorna valor. Usa a palavra `'void'` antes do nome (ex.: `public void ligar();`);
- Método com retorno: declara o tipo de retorno antes do nome + usa `'return'` no corpo;
- Exemplos de retorno: `int`, `float`, `String`, `boolean` — o retorno deve corresponder ao tipo declarado.
- Comentários: `'//'` para linha única. Tudo após `'//'` é ignorado pelo compilador — serve como documentação.
- Boas práticas: comente o propósito de cada método para facilitar manutenção e leitura do código.

void vs. retorno: comparação lado a lado

```
// Método SEM retorno (procedimento)
public void exibirDados() {
    System.out.println("Modelo: " + this.modelo);
    // Não tem return
}

// Método COM retorno (função)
public int calcularIdade(int anoAtual) {
    return anoAtual - this.ano; // Retorna int
}
```

void = ação sem resposta | tipo primitivo = ação que devolve um valor



5.3 Modificadores de Acesso

Encapsulamento na prática: public, private, protected, default

O que são modificadores de acesso?

- Ferramentas que garantem o encapsulamento na linguagem *Java*.
- Definem quem tem direito de acessar diretamente atributos ou métodos.
- Quatro principais: `public`, `private`, `protected` e *default* (sem palavra-chave).
- `public`: acessível de qualquer lugar — classes, pacotes, subclasses.
- `private`: acessível apenas dentro da própria classe — máxima restrição.
- `protected`: acessível no mesmo pacote + subclasses em pacotes diferentes.
- *default* (sem modificador): acessível apenas dentro do mesmo pacote.

Tabela de modificadores de acesso

Escopo	<code>public</code>	<code>protected</code>	<code>default</code>	<code>private</code>
Mesma classe	SIM	SIM	SIM	SIM
Mesmo pacote	SIM	SIM	SIM	NÃO
Pacote diferente (subclasse)	SIM	SIM	NÃO	NÃO
Pacote diferente (sem subclasse)	SIM	NÃO	NÃO	NÃO

`public` = acesso total | `private` = apenas dentro da classe | `protected` = pacote + subclasses | `default` = apenas pacote

Dinâmica 3 — Aplicando Modificadores

- Em duplas, considere a classe `ContaBancaria` vista na Aula 3.
- Reescreva os atributos e métodos agora com os modificadores *Java* corretos:
- `saldo` → `private` (protegido), `titular` → `private`, `depositar()` → `public`, `sacar()` → `public`.
- `validarSenha()` → `private`, `consultarSaldo()` → `public`.
- Pergunta: como acessar '`saldo`' de fora da classe se ele é `private`? (dica: *getters/setters*)
- Tempo: 5 minutos.

Respostas — Dinâmica 3

- `private float saldo;`
`private String titular;`
 - → protegidos contra acesso externo direto.
 - `public void depositar(float valor) {`
`this.saldo += valor;`
`}`
 - → público, qualquer classe pode chamar.
 - Padrão *getter/setter*: métodos públicos que controlam acesso a atributos privados — encapsulamento na prática!
- `public float consultarSaldo() {`
`return this.saldo;`
`}`
 - → *getter* público para ler o saldo.
 - `private boolean validarSenha(String s){`
`...`
`}`
 - → interno, só a própria classe usa.



5.4 Construtores Múltiplos

Polimorfismo na prática: mesmo nome, parâmetros diferentes

O que são construtores múltiplos?

- Método construtor: tem o mesmo nome da classe, inicializa o objeto ao instanciá-lo;
- Construtores múltiplos = mesmo nome, mas com parâmetros diferentes (sobrecarga);
- Exemplificam polimorfismo: dependendo dos parâmetros passados, um construtor diferente é chamado;
- Cada construtor pode inicializar o objeto de forma diferente conforme o contexto;
- Permitem flexibilidade: criar objeto vazio, com dados parciais ou com todos os dados.

Exemplo: classe Aviao com 3 construtores

```
public class Aviao {  
    private String empresa;  
    private String modelo;  
  
    public Aviao() {           // Sem parâmetros  
        System.out.println("E vruu!");  
    }  
    public Aviao(int x, int y) { // 2 inteiros  
        System.out.println("Resultado = " + (x+y));  
    }  
    public Aviao(String empresa, String modelo) { // 2 Strings  
        this.empresa = empresa;  
        this.modelo = modelo;  
    }  
}
```

3 construtores com mesmo nome 'Aviao', mas parâmetros diferentes = polimorfismo

Dinâmica 4 — Criando Construtores Múltiplos

- Individualmente, crie a classe 'Produto' com 3 construtores diferentes:
 - Construtor 1: sem parâmetros — imprime 'Produto genérico criado'.
 - Construtor 2: recebe (`String nome`) — inicializa apenas o `nome`.
 - Construtor 3: recebe (`String nome, float preco, int estoque`) — inicializa tudo.
- Escreva também uma classe 'Loja' com método `main` que instancie 3 objetos Produto, um para cada construtor.
- Tempo: 7 minutos.

Exemplo de resposta — Dinâmica 4

- ```
public Produto() {

 System.out.println("Produto genérico criado");

}
```
- ```
public Produto(String nome) {  
  
    this.nome = nome;  
  
}
```
- ```
public Produto(String nome, float preco, int estoque) {
 this.nome = nome;
 this.preco = preco;
 this.estoque = estoque;
}
```

Na classe Loja:

- ```
Produto p1 = new Produto();
```
- ```
Produto p2 = new Produto("Caneta");
```
- ```
Produto p3 = new Produto("Mouse", 49.90f, 100);
```

Cada `'new Produto(...)'` chama um construtor diferente — isso é polimorfismo via sobrecarga.



5.5 Superclasse e Subclasse

Herança em Java com a palavra-chave extends

Herança em Java: extends

- Para criar uma subclasse, usa-se a palavra-chave '`extends`' na declaração;
- Exemplo: `public class Cachorro extends Animal { }` → Cachorro herda tudo de `Animal`;
- A superclasse `Animal` tem atributos genéricos: `especie`, `tamanho`, `cor` (`protected`);
- A subclasse `Cachorro` herda tudo e pode adicionar atributos próprios (ex.: `raca`, `adestrado`);
- Modificador `protected` nos atributos da superclasse permite acesso pelas subclasses;
- Resultado: reaproveitamento de código + organização hierárquica.

Dinâmica 5 — Herança Completa em *Java*

- Em duplas, criem uma hierarquia de herança em *Java* para um sistema de RH:
- Superclasse 'Funcionario': atributos `protected` (`nome`, `cpf`, `salario`) + método `calcularBonificacao()`.
- Subclasse 'Gerente': `extends` `Funcionario` + atributo `setor` + sobreescreve `calcularBonificacao()` (10%).
- Subclasse 'Estagiario': `extends` `Funcionario` + atributo `faculdade` + sobreescreve `calcularBonificacao()` (3%).
- Escrevam o código completo no papel ou caderno.
- Tempo: 8 minutos. Apresentação: 2 minutos por dupla.

Exemplo de resposta — Dinâmica 5

```
public class Funcionario {  
    protected String nome;  
    protected float salario;  
  
    public float calcularBonificacao() {  
        return salario * 0.05f;  
    }  
}
```

```
public class Gerente extends Funcionario {  
    private String setor;  
    public float calcularBonificacao() {  
        return salario * 0.10f;  
    }  
}  
  
public class Estagiario extends Funcionario  
{  
    private String faculdade;  
    public float calcularBonificacao() {  
        return salario * 0.03f;  
    }  
}
```

- Aqui temos herança (`extends`) + polimorfismo (mesmo método, comportamento diferente) + encapsulamento (`private/protected`).
- Os 4 princípios da OO trabalhando juntos na prática!

"Java une desempenho e portabilidade graças à JVM: escreva uma vez, rode em qualquer lugar."

Pontos-chave da aula

- *Java*: sintaxe baseada em *C*, modelo híbrido compilado+interpretado via *JVM*;
- Classes com letra maiúscula, escopo em `{ }`, método `main` como ponto de entrada.
- Métodos `void` (sem retorno) *vs.* métodos com retorno (`int`, `String`, `boolean...`).
- Modificadores: **`public`**, **`private`**, **`protected`**, *default* — encapsulamento na prática.
- Construtores múltiplos = polimorfismo. **`extends`** = herança entre classes.